



**UNIVERSIDAD LATINA
DE COSTA RICA**

POWERED BY Arizona State University

**UNIVERSIDAD LATINA DE COSTA RICA
SEDE SANTA CRUZ**

**CURSO:
BASES DE DATOS II**

**TEMA:
DOCUMENTO FINAL – MATCHCONTROL**

**PROFESOR:
DAVID ACUÑA MORA**

**ESTUDIANTE:
SOFIA VARGAS ESPINOZA
JENDRY LINNETH MURILLO PÉREZ**

I CUATRIMESTRE – 2026

Table of Contents

Resumen	4
Introducción	4
Justificación	5
Problemas de la gestión manual de torneos	6
Valor académico del proyecto	6
Objetivos del Proyecto	7
Objetivo General	7
Objetivos Específicos	7
Descripción General del Sistema	8
Diferenciación frente a soluciones existentes	8
Alcance y limitaciones	9
Arquitectura del Sistema	9
Tecnologías utilizadas	10
Modelo de Base de Datos	10
Módulos funcionales	11
Requerimientos Funcionales	11
RF-01: Gestión de Usuarios	11
RF-03: Inscripciones	12
RF-04: Gestión de Partidas	13
RF-05: Rankings y Estadísticas	14
Stored Procedures y Funciones	15
Stored Procedures principales por módulo	15
Funciones escalares	17
Vistas operativas	17
Triggers de Automatización	18
Índices y Optimización	19
Índices por tabla	19
Impacto en rendimiento	20
Análisis de Concurrencia	21
Gestión de deadlocks	22

Estrategia de Seguridad	22
Control de acceso basado en roles (RBAC)	22
Autenticación y protección de credenciales	23
Protección contra inyección SQL	23
Implementación del Backend.....	24
Estructura del proyecto	24
Patrón módulo / controlador / servicio	25
Autenticación y autorización JWT.....	25
Endpoints principales de la API	26
Manejo de errores	27
Implementación del Frontend.....	27
Estructura del proyecto	28
Routing y control de acceso por rol.....	28
Comunicación con el API	29
Flujos de usuario principales	29
Conclusiones	30
Anexos	32
Diagrama Entidad-Relación	32
Diagrama Relacional	32

Resumen

MatchControl es un sistema web completo de gestión de torneos multicategoría desarrollado como proyecto final del curso de Bases de Datos II. El sistema surge en respuesta a la problemática real que enfrentan organizadores de torneos estudiantiles, deportivos, académicos y gamer, quienes actualmente recurren a métodos manuales propensos a errores: hojas de cálculo, grupos de mensajería instantánea y aplicaciones no especializadas.

La solución implementada abarca tres capas completamente integradas. En la capa de persistencia se construyó una base de datos en SQL Server 2019 compuesta por 20 tablas normalizadas en Tercera Forma Normal (3FN), más de 60 procedimientos almacenados parametrizados, 7 triggers de automatización, 4 funciones escalares, 6 vistas operativas y más de 30 índices NONCLUSTERED estratégicos. En la capa de lógica de negocio se desarrolló una API RESTful utilizando NestJS con TypeScript, que expone todos los módulos del sistema bajo un esquema de autenticación JWT y control de acceso basado en roles (RBAC) de cuatro niveles. Finalmente, la capa de presentación fue construida con Vue 3, Vite y Tailwind CSS, ofreciendo una interfaz responsiva y moderna que cubre todos los flujos de usuario del sistema.

El proyecto integra de manera práctica y aplicada los conceptos avanzados del curso: diseño y normalización de bases de datos relacionales, programación en T-SQL, gestión de concurrencia mediante niveles de aislamiento de transacciones, estrategias de seguridad, optimización de consultas mediante índices y análisis de planes de ejecución.

Introducción

La organización de competencias —ya sean torneos deportivos universitarios, ligas de videojuegos entre estudiantes o campeonatos académicos— representa un reto logístico considerable para quienes asumen el rol de organizadores. En la práctica, este proceso involucra la gestión simultánea de inscripciones, la creación de calendarios de partidos, el registro de resultados en tiempo real y la actualización de tablas de posiciones, todo ello con múltiples actores operando al mismo tiempo y expectativas de inmediatez y transparencia por parte de los participantes.

La realidad actual es que la mayoría de estos procesos se gestionan de forma manual. Los organizadores crean hojas de cálculo para llevar el control de inscritos, publican los resultados en

grupos de WhatsApp o Instagram, y actualizan las posiciones de forma manual tras cada jornada. Este enfoque introduce errores inevitables: participantes inscritos dos veces, cupos que se exceden, resultados mal registrados y una total ausencia de trazabilidad sobre quién modificó qué dato y cuándo.

MatchControl nace como respuesta directa a esta problemática. El sistema centraliza y automatiza los procesos críticos de gestión de torneos, garantizando la integridad de la información mediante las capacidades del motor de base de datos SQL Server, y ofreciendo una experiencia de usuario moderna e intuitiva para todos los actores involucrados: administradores, organizadores, árbitros y participantes.

Desde la perspectiva académica, el proyecto integra de manera transversal los contenidos del curso de Bases de Datos II, incluyendo diseño y normalización de esquemas relacionales, programación avanzada en T-SQL, control de concurrencia, estrategias de seguridad en base de datos y optimización de rendimiento mediante índices. La elección de SQL Server como motor central no es accidental: su ecosistema de stored procedures, triggers y funciones escalares permite delegar la lógica crítica del negocio al propio motor de base de datos, garantizando consistencia independientemente de la capa de aplicación que realice las modificaciones.

Justificación

La implementación de un sistema web para la gestión de torneos es necesaria debido a los problemas que presentan los métodos manuales actuales, tales como errores en el registro de participantes, dificultad para controlar cupos, falta de transparencia en los resultados y desorganización de la información.

MatchControl permite automatizar los procesos principales del torneo, garantizando la integridad de los datos mediante SQL Server. Además, incorpora control de accesos por roles, auditoría de acciones y optimización de consultas.

Este proyecto fortalece las habilidades técnicas del estudiante y permite aplicar conocimientos en un entorno práctico.

Problemas de la gestión manual de torneos

La gestión manual de torneos presenta cinco problemas estructurales que MatchControl resuelve de forma sistemática:

- Inscripciones duplicadas: sin validación automática, un participante puede registrarse múltiples veces o un equipo puede inscribirse en más torneos de los permitidos. El sistema resuelve esto mediante constraints de unicidad a nivel de base de datos y validaciones dentro de los stored procedures.
- Falta de control de cupos: la verificación manual del cupo máximo es lenta y propensa a condiciones de carrera cuando múltiples usuarios intentan inscribirse simultáneamente. MatchControl implementa el nivel de aislamiento SERIALIZABLE en el proceso de inscripción para eliminar este riesgo.
- Resultados poco confiables: la introducción manual de scores genera errores de transcripción y no deja historial verificable de cuándo fue registrado el resultado ni por quién. Los triggers de auditoría del sistema resuelven esta ausencia de trazabilidad.
- Información desorganizada: el uso de múltiples herramientas dispersas (hojas de cálculo, grupos de mensajería, formularios web) imposibilita la generación de reportes consolidados y obliga a la reconciliación manual de datos.
- Ausencia de control de acceso: sin un sistema de roles, cualquier persona puede modificar resultados o inscripciones, comprometiendo la integridad del torneo. MatchControl implementa RBAC con cuatro niveles de acceso claramente definidos.

Valor académico del proyecto

Más allá de la utilidad práctica, el proyecto permite aplicar en un contexto real los conceptos más relevantes del curso. El diseño de 20 tablas en 3FN exige analizar dependencias funcionales y transitividad. La implementación de más de 60 stored procedures desarrolla habilidades de programación T-SQL avanzada. El análisis de concurrencia obliga a comprender en profundidad cómo SQL Server gestiona el aislamiento de transacciones. Los índices NONCLUSTERED estratégicos y el análisis de planes de ejecución conectan la teoría de optimización con resultados medibles y verificables.

El proyecto también tiene proyección futura: una vez finalizada la etapa académica, puede evolucionar hacia una plataforma real con modelo freemium, incorporando procesamiento de pagos de inscripciones, notificaciones en tiempo real mediante WebSockets, módulo de análisis predictivo de resultados y aplicación móvil nativa consumiendo la misma API RESTful.

Objetivos del Proyecto

Objetivo General

Desarrollar un sistema web completo —base de datos, backend y frontend— que permita gestionar torneos multicategoría de manera eficiente, segura y automatizada, aplicando los conceptos avanzados del curso de Bases de Datos II sobre SQL Server junto con NestJS, TypeScript, Vue 3 y tecnologías web modernas.

Objetivos Específicos

- Diseñar una base de datos normalizada en 3FN con 20 tablas distribuidas en 7 módulos funcionales, garantizando integridad referencial mediante claves foráneas nombradas explícitamente.
- Implementar más de 60 stored procedures parametrizados con el patrón SET NOCOUNT ON / BEGIN TRY-CATCH / transacciones explícitas, cubriendo CRUD completo para todos los módulos del sistema.
- Automatizar inscripciones, validaciones y posiciones mediante 7 triggers de base de datos que operan independientemente de la capa de aplicación.
- Construir una API RESTful con NestJS y TypeScript que exponga todos los módulos del sistema con validación de inputs, manejo estructurado de errores y documentación de endpoints.
- Implementar autenticación JWT con RBAC de 4 niveles (Administrador, Organizador, Árbitro, Participante) y guards de autorización por ruta en el backend.
- Desarrollar una interfaz web responsiva con Vue 3, Vite y Tailwind CSS que cubra todos los flujos de usuario definidos en los requerimientos funcionales.

- Implementar más de 30 índices NONCLUSTERED estratégicos diseñados en función de los patrones de consulta reales del sistema y documentar su impacto en los planes de ejecución.
- Gestionar correctamente los escenarios de concurrencia críticos mediante niveles de aislamiento de transacciones apropiados para cada módulo del sistema.

Descripción General del Sistema

MatchControl es una plataforma web full stack que centraliza y automatiza la gestión integral de torneos multicategoría. El sistema soporta cuatro formatos de competencia: Eliminación Directa, Round Robin, Grupos y Sistema Suizo, cada uno con sus propias reglas de enfrentamiento y cálculo de posiciones.

El sistema está diseñado para cuatro tipos de actores, cada uno con su propio espacio funcional dentro de la plataforma. El Administrador tiene control total sobre usuarios, organizaciones y la configuración global del sistema, accediendo desde el panel administrativo. El Organizador crea y administra torneos, gestiona las fases y grupos, aprueba o rechaza inscripciones y supervisa el desarrollo del torneo desde su dashboard. El Árbitro tiene acceso exclusivo al panel de registro de resultados, donde puede ingresar scores y el detalle por set o mapa de cada partido que le ha sido asignado. El Participante accede al portal público para inscribirse en torneos, consultar su historial, revisar las tablas de posiciones y gestionar su perfil.

Una característica fundamental del diseño es que toda la lógica crítica de negocio — validaciones de cupo, cálculo de posiciones, control de concurrencia, auditoría— reside en la base de datos, no en la capa de aplicación. Esto garantiza que la integridad del sistema se preserve independientemente de si los datos son modificados a través de la API, de herramientas administrativas directas o de scripts de mantenimiento.

Diferenciación frente a soluciones existentes

El mercado cuenta con herramientas como Challonge, Toornament y Battlefy, orientadas principalmente a torneos de videojuegos con interfaces simplificadas. MatchControl se diferencia en tres aspectos clave: primero, su arquitectura de base de datos robusta garantiza consistencia

incluso en escenarios de alta concurrencia; segundo, el sistema de auditoría completo ofrece trazabilidad total de todas las acciones críticas; tercero, el RBAC de cuatro niveles permite adaptarlo a contextos institucionales como universidades, ligas deportivas o comunidades gamer con estructuras organizativas complejas.

Alcance y limitaciones

El sistema permite gestionar el ciclo de vida completo de un torneo: creación, inscripciones, fases, grupos, partidos, sets, resultados, posiciones, sanciones y notificaciones. Como limitación deliberada para la versión académica, el sistema no incluye procesamiento de pagos de inscripciones ni está configurado para un entorno de producción con alta disponibilidad. Estas funcionalidades están identificadas como trabajo futuro.

Arquitectura del Sistema

MatchControl sigue una arquitectura de tres capas con separación clara de responsabilidades, donde cada capa se comunica únicamente con la capa adyacente mediante interfaces bien definidas. Esta decisión arquitectónica garantiza que los cambios en una capa no afecten directamente a las demás, facilitando el mantenimiento y la evolución del sistema.

Aspecto	Presentación	Lógica de Negocio (NestJS)	Persistencia (SQL Server)
Tecnología principal	Vue 3 + Vite + Tailwind	NestJS + TypeScript	SQL Server 2019
Estructura	Módulos por dominio	Módulos por entidad	20 tablas en 7 módulos
Comunicación	HTTP → API REST	Guards JWT + servicios	Stored procedures
Autenticación	Token JWT en cliente	Verificación guard global	---
Seguridad	Guards de ruta por rol	RBAC por endpoint	Sin SQL dinámico

Automatización	---	---	7 triggers
----------------	-----	-----	------------

Tecnologías utilizadas

Capa	Tecnología	Responsabilidad
Presentación	Vue 3, Vite, Tailwind CSS	Interfaz de usuario responsiva, SPA, routing por rol
Lógica de negocio	NestJS, TypeScript, JWT	API RESTful, autenticación, autorización RBAC, validación de inputs
Persistencia	SQL Server 2019	20 tablas, 60+ SPs, 7 triggers, 30+ índices NONCLUSTERED
Control de versiones	Git / Github	Repositorio único con monorepo: Backend-MatchControl + matchcontrol-front

Modelo de Base de Datos

La base de datos MatchControl está compuesta por 20 tablas organizadas en 7 módulos funcionales, todas normalizadas en Tercera Forma Normal (3FN). Esto implica que cada tabla cumple con las propiedades de la Primera Forma Normal (todos los atributos son atómicos), la Segunda Forma Normal (no existen dependencias parciales de la clave primaria) y la Tercera Forma Normal (no existen dependencias transitivas entre atributos no clave). La normalización garantiza la eliminación de redundancias, facilita el mantenimiento del esquema y reduce la probabilidad de anomalías de inserción, actualización y eliminación.

Módulos funcionales

Requerimientos Funcionales

RF-01: Gestión de Usuarios

ID	Caso de Uso	Actor	SP Asociado	Resultado Esperado
RF-01.1	Registrar usuario	Admin	sp_InsertarUsuario	Usuario creado con hash bcrypt. Trigger registra en Auditoría.
RF-01.2	Consultar usuarios	Admin	sp_ObtenerUsuario	Lista de usuarios activos con rol y organización.
RF-01.3	Actualizar datos	Admin	sp_ActualizarUsuario	Nombre y email actualizados.
RF-01.4	Desactivar cuenta	Admin	sp_EliminarUsuario	Estado=0 (Soft Delete). Historial preservado.
RF-01.5	Solicitar cambio de rol	Participante	sp_InsertarSolicitudRol	Solicitud registrada en Solicitud_Rol.
RF-01.6	Procesar solicitud de rol	Admin	sp_ProcesarSolicitudRol	Rol actualizado si aprobado; notificación enviada.

RF-02: Gestión de Torneos

ID	Caso de Uso	Actor	SP Asociado	Resultado Esperado
RF-02.1	Crear torneo	Organizador	sp_InsertarTorneo	Torneo creado en estado Borrador. Valida MaxParticipantes > 0.
RF-02.2	Consultar detalle	Todos	sp_ConsultarTorneoDetalle	Torneo con nombre de disciplina y organización.

RF-02.3	Cambiar estado	Organizador/Adm in	sp_ActualizarTorneoEstado	Estado actualizado: Borrador→Inscripciones→En Curso→Finalizado.
RF-02.4	Eliminar torneo	Admin	sp_EliminarTorneo	Eliminación en cascada de Fases, Grupos, Matches y Posiciones. Trigger audita.
RF-02.5	Gestionar fases	Organizador	sp_InsertarFase / sp_ActualizarFase	Fases creadas con tipo (Grupos, Eliminación, Round Robin) y orden.
RF-02.6	Gestionar grupos	Organizador	sp_InsertarGrupo / sp_ObtenerGrupo	Grupos creados dentro de una fase.

RF-03: Inscripciones

ID	Caso de Uso	Actor	SP Asociado	Resultado Esperado
RF-03.1	Inscribir usuario individual	Participante	sp_InscribirParticipante	Registro en Participante (Id_Equipo=NULL). Constraint CHK_Unico_Tipo.
RF-03.2	Inscribir equipo	Capitán	sp_InscribirParticipante	Registro con Id_Equipo (Id_Usuario=NULL).
RF-03.3	Consultar inscripciones	Organizador	sp_ObtenerParticipante	Lista con usuario/equipo, estado y fecha.
RF-03.4	Aprobar / Rechazar	Organizador	sp_ActualizarInscripcion	Estado cambiado a Aceptado/Rechazado. Trigger crea fila en

				Posiciones si Aceptado.
RF-03.5	Crear equipo	Participante	sp_InsertarEquipo	Equipo creado con capitán, siglas y logo opcional.
RF-03.6	Agregar jugador	Capitán	sp_AgregarJugadorEquipo	Jugador agregado a Equipo_Jugador. Valida no duplicados.

RF-04: Gestión de Partidas

ID	Caso de Uso	Actor	SP Asociado	Resultado Esperado
RF-04.1	Crear match	Organizador	sp_InsertarMatch	Partido en estado Programado dentro de una fase.
RF-04.2	Asignar participantes	Organizador/Árbitro	sp_AsignarParticipanteMatch	Lado 1 y Lado 2 asignados al match.
RF-04.3	Registrar resultado	Organizador/Árbitro	sp_RegistrarResultadoMatch	Scores actualizados; estado del match pasa a Finalizado.
RF-04.4	Registrar sets	Organizador/Árbitro	sp_InsertarSet	Detalle por mapa/ronda guardado en Match_Set.

RF-04.5	Consultar partidas	Todos	sp_ObtenerMatch	Lista con árbitro, fase, grupo y torneo.
RF-04.6	Registrar sanción	Organizador	sp_InsertarSancion	Sanción (Advertencia, Suspensión, Descalificación) registrada.

RF-05: Rankings y Estadísticas

ID	Caso de Uso	Actor	SP / Vista	Resultado Esperado
RF-05.1	Consultar tabla de posiciones	Todos	sp_ConsultarPosiciones / sp_ObtenerTablaPosiciones	Ranking ordenado por Puntos DESC, Diferencia DESC.
RF-05.2	Reporte general de resultados	Admin/Organizador	vw_ReporteGeneralResultados	Historial de partidos con scores y ganadores por torneo.
RF-05.3	Inscripciones pendientes	Organizador	vw_InscripcionesPendientes	Lista de participantes en estado Pendiente.
RF-05.4	Estadísticas de equipos	Admin	vw_EstadisticasEquipos	Torneos jugados y victorias

				totales por equipo.
RF-05.5	Calendario de próximos matches	Todos	vw_CalendarioProximosMatches	Partidos en estado Programado con torneo y fase.
RF-05.6	Logs de auditoría	Admin	vw_LogsAuditoriaReciente	100 últimas acciones del sistema.
RF-05.7	Vista de usuarios detallados	Admin	vw_UsuariosDetallados	Usuarios con rol, organización y estado legible.

Stored Procedures y Funciones

MatchControl implementa más de 60 stored procedures con un patrón uniforme: SET NOCOUNT ON, manejo de errores con BEGIN TRY/CATCH y THROW, y transacciones explícitas con COMMIT/ROLLBACK. Ningún módulo construye SQL dinámico; toda la lógica de acceso a datos reside en los procedimientos.

Stored Procedures principales por módulo

Módulo	Stored Procedures	Descripción
Usuarios	sp_InsertarUsuario, sp_ObtenerUsuario, sp_ActualizarUsuario, sp_EliminarUsuario	CRUD completo. Soft Delete. Valida nickname y email únicos antes de insertar.

Torneos	sp_InsertarTorneo, sp_ConsultarTorneoDetalle, sp_ActualizarTorneoEstado, sp_EliminarTorneo	Crea en estado Borrador. Eliminación en cascada de hasta 7 tablas en una transacción.
Fases y Grupos	sp_InsertarFase, sp_ActualizarFase, sp_InsertarGrupo, sp_ActualizarGrupo	Gestión de etapas y subdivisiones del torneo con orden definido.
Participantes	sp_InscribirParticipante, sp_ObtenerParticipante, sp_ActualizarInscripcion, sp_EliminarParticipante	Inscripción individual o de equipo. Valida constraint CHK_Unico_Tipo.
Matches	sp_InsertarMatch, sp_ObtenerMatch, sp_RegistrarResultadoMatch, sp_AsignarParticipanteMatch, sp_InsertarSet	Gestión completa de partidos. sp_RegistrarResultadoMatch cierra el match y dispara triggers.
Posiciones	sp_ConsultarPosiciones, sp_ObtenerTablaPosiciones	Ranking por torneo/fase ordenado por Puntos DESC y diferencia de score como desempate.
Solicitudes de Rol	sp_InsertarSolicitudRol, sp_ObtenerSolicitudesRol, sp_ProcesarSolicitudRol	Flujo completo de solicitud, aprobación y actualización atómica del rol del usuario.
Notificaciones	sp_InsertarNotificacion, sp_ObtenerNotificaciones, sp_MarcarNotificacionLeida	Mensajes internos con bandeja por usuario y estado de lectura.

Funciones escalares

Función	Retorna	Propósito
fn_ObtenerPuntosTotales	INT	Calcula puntos: (Ganados $\times$ PtsWin) + (Empatados $\times$ PtsDraw). Permite puntuación personalizada por torneo.
fn_ValidarCupoTorneo	BIT	Retorna 1 si hay cupos disponibles, 0 si el torneo está lleno.
fn_GetNombreParticipante	NVARCHAR(150)	Devuelve el nombre visible del participante. Fallback a "No Registrado" si no existe.
fn_CalcularDiferenciaScore	INT	Diferencia Score_Favor – Score_Contra para criterio de desempate.
fn_EstadoTorneoColor	NVARCHAR(20)	Retorna un color (Verde/Azul/Gris/Rojo) según el estado del torneo para el frontend.

Vistas operativas

Vista	Tablas involucradas	Propósito
vw_ReporteGeneralResultados	Match, Fase, Torneo, Match_Participante, Participante	Historial completo de partidos con scores y ganadores.
vw_InscripcionesPendientes	Participante, Torneo, Usuario	Lista participantes en estado Pendiente para el organizador.
vw_EstadisticasEquipos	Equipo, Participante, Posiciones	Torneos jugados y victorias totales por equipo.
vw_CalendarioProximosMatches	Match, Fase, Torneo	Partidos en estado Programado con fecha, ubicación y torneo.

vw_LogsAuditoriaReciente	Auditoria	TOP 100 últimas acciones del sistema ordenadas cronológicamente.
vw_UsuariosDetallados	Usuario, Rol, Organizacion	Usuarios activos con nombre de rol, organización y estado legible.

Triggers de Automatización

Los triggers garantizan la consistencia de los datos independientemente de qué capa o herramienta realice las modificaciones. Si un administrador edita datos directamente desde SQL Server Management Studio, los triggers se activan de la misma manera que si el cambio proviniera de la API, convirtiendo al motor de base de datos en el guardián definitivo de las reglas de negocio.

Trigger	Tabla	Evento	Propósito
trg_AuditoriaUsuarios	Usuario	AFTER INSERT	Registra en Auditoria cada nuevo usuario con nickname, rol y fecha.
trg_ControlCambiosScore	Match_Participante	AFTER UPDATE	Bloquea edición de scores si el partido está en estado Finalizado. Ejecuta ROLLBACK automático.
trg_AuditoriaEliminacionTorneo	Torneo	AFTER DELETE	Registra en Auditoria el nombre y formato del torneo eliminado antes de que desaparezca.
trg_AutoCrearPosicion	Participante	AFTER UPDATE	Al aceptar una inscripción, crea automáticamente la fila inicial en Posiciones con todos los contadores en 0.

trg_ValidarReglasSet	Match_Set	AFTER INSERT / UPDATE	Rechaza puntajes negativos e IDs de ganador inválidos. ROLLBACK si falla.
trg_CalcularPuntosAutomatica	Match_Participante	AFTER UPDATE	Al modificar Score_Final, actualiza PJ, PG, PE, PP, Puntos y scores en Posiciones.
trg_ActualizarTablaPosiciones	Match_Participante	AFTER INSERT	Al insertar un resultado nuevo, actualiza la fila de posiciones del participante afectado.

Índices y Optimización

MatchControl implementa más de 30 índices NONCLUSTERED idempotentes (IF NOT EXISTS), diseñados a partir del análisis de las consultas más frecuentes del sistema: login por email, tabla de posiciones, validación de cupos, inscripciones pendientes, calendario de partidos y logs de auditoría.

Índices por tabla

Tabla	Índice	Columnas clave	Justificación
Usuario	IX_Usuario_Rol	Nombre, Nickname, Email, Estado, Id_Rol	Optimiza login y JOINS en vw_UsuariosDetallados. Convierte Table Scan en Index Seek.
Torneo	IX_Torneo_Estado	Nombre, Id_Disciplina, Id_Org,	Cubre todos los filtros del listado público de torneos.

		Formato, Fecha_Inicio, Estado	
Match	IX_Match_Estado	Id_Fase, Id_Grupo, Fecha_Hora, Estado	Alimenta vw_CalendarioProximosMa tches. Reduce ~90% páginas leídas.
Match_Particip ante	IX_MatchPart_Match_Lad o	Id_Participante, Es_Ganador, Score_Final, Id_Match, Lado	Cobertura bidireccional para JOINS entre partidos y participantes.
Posiciones	IX_Posiciones_Fase_Punt os	Id_Grupo, Id_Participante, Score_Favor, Score_Contra, Id_Fase, Puntos	Elimina operador Sort del plan. Datos retornados en orden del índice.
Participante	IX_Participante_Torneo_E stado	Id_Usuario, Id_Equipo, Nombre, Id_Torneo, Estado_Inscripc ion	Optimiza vw_InscripcionesPendientes . Filtro selectivo por Estado.
Auditoria	IX_Auditoria_Fecha	Id_Usuario, Accion, Tabla, Fecha	Range Scan para TOP 100 sin ordenamiento completo de la tabla.

Impacto en rendimiento

Consulta crítica	Sin índice	Con índice	Mejora estimada
Login por email	Table Scan O(n)	Index Seek O(log n)	~95% reducción en lecturas lógicas

Tabla de posiciones	Sort + Hash Match	Index Scan ordenado	Elimina operador Sort del plan
Validar cupo de torneo	Full Scan Participante	Index Seek puntual	~80% reducción de filas evaluadas
Inscripciones pendientes	Clustered Index Scan	Seek por Estado	Filtro selectivo por Estado_Inscripcion
Calendario de partidos	Table Scan Match	Range Scan por fecha	~90% reducción para rangos de fecha

Análisis de Concurrency

Al ser una plataforma multiusuario, MatchControl gestiona cuatro escenarios críticos de acceso simultáneo. Cada uno aplica un nivel de aislamiento específico para garantizar la integridad sin sacrificar rendimiento innecesariamente.

Escenario	Riesgo	Solución implementada	Nivel de aislamiento
Inscripciones simultáneas	Race condition en cupo máximo	La lectura del cupo y la inserción son atómicas. Ninguna otra transacción puede insertar hasta que la actual termine.	SERIALIZABLE
Registro de resultados simultáneo	Conflictos en tabla Posiciones entre árbitros	Cada árbitro trabaja con vista consistente. ROWLOCK en actualizaciones de Posiciones minimiza contención.	SNAPSHOT + ROWLOCK
Aprobación simultánea de inscripciones	Duplicados en Posiciones por doble aprobación	Cláusula NOT EXISTS en trg_AutoCrearPosicion previene inserción doble. SP de aprobación con SERIALIZABLE.	SERIALIZABLE

Eliminación de torneo en uso	Deadlocks o violación de integridad referencial	Soft Delete en operaciones normales. sp_EliminarTorneo adquiere bloqueos en orden fijo para evitar deadlocks.	READ COMMITTED
------------------------------	---	---	----------------

Gestión de deadlocks

El escenario de deadlock más probable ocurre entre el árbitro (que bloquea Match_Participante e intenta actualizar Posiciones) y el trigger de cálculo de puntos (que tiene bloqueada Posiciones e intenta leer Match_Participante). Las estrategias de prevención son:

- Orden de acceso consistente a las tablas: siempre Match → Match_Participante → Posiciones.
- Transacciones lo más cortas posible para reducir el tiempo de retención de bloqueos.
- Índices covering que disminuyen la duración de los bloqueos al necesitar menos páginas.
- SET LOCK_TIMEOUT en el backend con retry logic exponencial ante timeouts.

Estrategia de Seguridad

La seguridad se implementa en múltiples capas bajo el principio de defensa en profundidad: si una capa es vulnerada, las inferiores mantienen la protección.

Control de acceso basado en roles (RBAC)

Rol	Permisos	Restricciones
Administrador	Control total: usuarios, organizaciones, configuración, auditoría.	No puede modificar registros de Auditoría. Solo desactivar usuarios (Soft Delete).
Organizador	Crear/editar sus torneos, aprobar inscripciones, gestionar fases.	Solo gestiona torneos donde Id_Creador = su Id_Usuario.

Árbitro	Registrar resultados y sets de partidos asignados.	Sin acceso a gestión de torneos, inscripciones ni datos de otros árbitros.
Participante	Inscribirse, consultar resultados, rankings e historial propio.	Solo lectura en estadísticas. No modifica datos de otros participantes.

Autenticación y protección de credenciales

Las contraseñas se almacenan como hash bcrypt (coste 12), irreversible y salteado. Al autenticarse exitosamente, el servidor genera un JWT firmado con HS256 (clave de 256 bits) con expiración de 8 horas, que incluye en el payload el Id_Usuario, el rol y el Id_Organizacion. Tras cinco intentos fallidos consecutivos, la cuenta se suspende temporalmente (Estado=0) y el evento queda registrado en Auditoria.

Protección contra inyección SQL

Todo acceso a datos se realiza exclusivamente mediante stored procedures con parámetros tipados. El backend nunca construye cadenas SQL dinámicas; todos los inputs se pasan como parámetros del tipo correcto a través del driver, garantizando que el motor los trate como valores literales y nunca como código ejecutable.

Implementación del Backend

El backend de MatchControl está construido con NestJS, un framework progresivo para Node.js que utiliza TypeScript y adopta patrones arquitectónicos de módulos, controladores, servicios e inyección de dependencias. Esta elección garantiza un código altamente organizado con tipado estático, lo que reduce errores en tiempo de ejecución y facilita el mantenimiento y la escalabilidad del sistema.

Estructura del proyecto

El repositorio de backend se organiza dentro de la carpeta Backend-MatchControl. La carpeta src contiene un módulo independiente por cada entidad del sistema, siguiendo una estructura uniforme en todos los módulos. La carpeta dist almacena el JavaScript compilado para producción, y la carpeta docs mantiene la documentación técnica del proyecto.

Elemento	Descripción
src/[modulo]/	Un módulo por entidad: auditoria, auth, categoria, configuracion-sistema, disciplina, equipo, equipo_jugador, fase, grupo, match, match_participante, match_set, notificacion, organizacion, participante, posiciones, rol, sancion, solicitud_rol, torneo, usuario.
controller.ts	Manejador de rutas HTTP. Recibe la solicitud, aplica guards y decoradores, y delega la lógica al servicio correspondiente.
service.ts	Lógica de negocio y punto de llamada a los stored procedures de SQL Server. Nunca construye SQL dinámico.
module.ts	Registro NestJS del módulo. Declara controladores, servicios y dependencias inyectadas.
DTOs	Data Transfer Objects opcionales para validación y sanitización de inputs con class-validator antes de llegar al controlador.
main.ts	Punto de entrada. Inicializa la aplicación NestJS, configura CORS, registra guards globales y arranca el servidor en el puerto configurado.
app.module.ts	Módulo raíz. Importa y registra todos los módulos del sistema.

dist/	JavaScript compilado listo para producción. Generado con tsc desde los archivos TypeScript.
docs/	Documentación técnica: API.md (endpoints), DATABASE.md (esquema BD), STRUCTURE.md y MatchControl.md.

Patrón módulo / controlador / servicio

Cada módulo del sistema sigue el mismo patrón arquitectónico. El controlador recibe la solicitud HTTP, verifica que el guard JWT haya autenticado al usuario y que el guard de roles autorice la operación, extrae y valida los parámetros mediante DTOs, y llama al método correspondiente del servicio. El servicio ejecuta el stored procedure a través del driver de SQL Server, mapea el resultado y lo retorna al controlador, que lo envía como respuesta HTTP con el código de estado apropiado. Este flujo garantiza que ninguna lógica de negocio resida en los controladores y que ningún SQL dinámico se construya en los servicios.

Autenticación y autorización JWT

El sistema implementa autenticación stateless basada en JSON Web Tokens. El flujo completo opera de la siguiente manera:

#	Paso	Detalle técnico
1	Envío de credenciales	Cliente envía email + contraseña al endpoint POST /auth/login.
2	Consulta en BD	El servicio de auth ejecuta sp_ObtenerUsuario para recuperar el hash almacenado en Password_Hash.
3	Verificación hash	bcryptjs compara la contraseña enviada con el hash. Si no coincide → HTTP 401 Unauthorized.
4	Generación del token	Se genera un JWT firmado con HS256 usando una clave secreta de 256 bits. Payload: { Id_Usuario, rol, Id_Organizacion }. Expiración: 8 horas.

5	Retorno al cliente	El token se envía al cliente junto con los datos básicos del usuario (id, nombre).
6	Uso en solicitudes	El cliente incluye el token en el header Authorization: Bearer <token> en cada solicitud posterior.
7	Validación en guard	El guard global de NestJS verifica la firma y la vigencia del token. Si es inválido o expirado → HTTP 401.
8	Autorización por rol	El guard de roles comprueba que el rol incluido en el payload tenga permisos para el endpoint solicitado. Si no → HTTP 403.

El sistema también implementa bloqueo temporal de cuentas: tras cinco intentos fallidos consecutivos de autenticación, el backend actualiza Estado=0 en la tabla Usuario y registra el evento en Auditoria. La cuenta queda suspendida hasta que un administrador la reactive.

Endpoints principales de la API

Método	Endpoint	Rol requerido	SP ejecutado
POST	/auth/login	Ninguno	sp_ObtenerUsuario
POST	/auth/register	Ninguno	sp_InsertarUsuario
GET	/users	Admin	sp_ObtenerUsuario
PUT	/users/:id	Admin	sp_ActualizarUsuario
DELETE	/users/:id	Admin	sp_EliminarUsuario (Soft Delete)
GET	/tournaments	Autenticado	sp_ConsultarTorneoDetalle
POST	/tournaments	Organizador / Admin	sp_InsertarTorneo
PUT	/tournaments/:id/status	Organizador / Admin	sp_ActualizarTorneoEstado
POST	/participants	Autenticado	sp_InscribirParticipante
PUT	/participants/:id	Organizador / Admin	sp_ActualizarInscripcion
PUT	/matches/:id/result	Árbitro / Admin	sp_RegistrarResultadoMatch

POST	/matches/:id/sets	Árbitro / Admin	sp_InsertarSet
GET	/stats/standings/:tournamentId	Autenticado	sp_ConsultarPosiciones

Manejo de errores

El manejador de excepciones global del backend captura todos los errores no controlados. Los errores THROW de los stored procedures (rango 50000–59999) se mapean a respuestas HTTP con el mensaje original de SQL Server, proporcionando feedback descriptivo al cliente sin exponer detalles internos.

Código SQL	HTTP Status	Descripción
50030	400 Bad Request	Nickname ya está en uso.
50031	400 Bad Request	Email ya está en uso.
50001	400 Bad Request	Max_Participantes debe ser mayor que 0.
50040	400 Bad Request	El jugador ya pertenece a este equipo.
50050	400 Bad Request	Solicitud de rol no encontrada.
50060	400 Bad Request	No se puede eliminar: la disciplina tiene torneos asociados.
50070	400 Bad Request	El equipo no existe.
50080	400 Bad Request	El participante no está registrado en este match.
—	403 Forbidden	Rol insuficiente para la operación solicitada.
—	500 Internal Server Error	Error inesperado. Mensaje genérico al cliente, detalle en logs internos.

Implementación del Frontend

El frontend de MatchControl es una Single Page Application (SPA) construida con Vue 3 utilizando la Composition API. Vite actúa como bundler y servidor de desarrollo, ofreciendo Hot Module Replacement (HMR) instantáneo durante el desarrollo y builds altamente optimizados para producción. Tailwind CSS provee un sistema de diseño utilitario que permite construir interfaces responsivas y consistentes sin escribir CSS personalizado. La estructura modular del

frontend replica exactamente la del backend: existe un módulo por cada entidad del sistema, manteniendo coherencia conceptual en todo el repositorio.

Estructura del proyecto

Elemento	Descripción
src/[modulo]/	Módulos por entidad: auditoria, auth, categoria, configuracion-sistema, disciplina, equipo, equipo_jugador, fase, grupo, match, match_participante, notificacion, organizacion, participante, posiciones, rol, sancion, solicitud_rol, torneo, usuario.
public/	Activos estáticos servidos directamente: favicon.svg, icons.svg.
index.html	Punto de entrada de la SPA. Contiene el div raíz donde Vue monta la aplicación.
vite.config.js	Configuración del bundler: alias de rutas, plugins de Vue, proxy del API para desarrollo local.
tailwind.config.js	Define la paleta de colores del sistema, tipografía y breakpoints responsivos.
eslint.config.js	Reglas de calidad y consistencia del código TypeScript/Vue.
package.json	Dependencias principales: vue, vue-router, vite, tailwindcss, axios.

Routing y control de acceso por rol

Vue Router gestiona la navegación entre vistas de la SPA. Cada ruta tiene definidos los roles que pueden acceder a ella mediante metadatos de ruta (meta.roles). Antes de renderizar cualquier página, un guard de navegación global verifica dos condiciones: que el usuario esté autenticado (token JWT válido en el cliente) y que su rol esté incluido en la lista de roles permitidos por esa ruta. Si alguna condición falla, el guard redirige al login o a una página de acceso denegado según corresponda. Esto garantiza que ningún usuario pueda acceder a vistas que no le corresponden, independientemente de si conoce la URL directa.

Comunicación con el API

Todas las llamadas HTTP al backend se realizan mediante un módulo centralizado que gestiona automáticamente la inyección del token JWT y el manejo uniforme de errores. Al momento de la autenticación exitosa, el token se almacena en el cliente y se incluye en el encabezado Authorization: Bearer <token> de cada solicitud subsiguiente. Si el servidor retorna HTTP 401, el módulo limpia el token almacenado y redirige automáticamente al login, evitando que el usuario quede en un estado inconsistente con un token expirado.

Aspecto	Implementación
Token JWT	Almacenado en el cliente al autenticarse. Incluido automáticamente en el header Authorization de cada solicitud.
Manejo de errores	Errores HTTP 400 muestran el mensaje original del servidor al usuario. HTTP 401 redirige al login. HTTP 403 muestra pantalla de acceso denegado.
Actualización de posiciones	Polling cada 30 segundos mediante setInterval. Actualiza la tabla de posiciones sin recargar la página completa.
Confirmaciones	Operaciones críticas (inscripción, registro de resultado, aprobación) muestran modal de confirmación antes de enviar la solicitud.

Flujos de usuario principales

Los cuatro flujos más críticos del sistema desde la perspectiva del usuario son los siguientes:

- **Flujo 1 - Inscripción en un torneo (Participante):** El participante navega al módulo de torneos, donde se listan los torneos en estado "Inscripciones". Al seleccionar uno y confirmar la inscripción, el frontend envía la solicitud al backend. Si el torneo tiene Requiere_Aprobacion=0, sp_InscribirParticipante crea la inscripción directamente en estado Aceptado y el trigger trg_AutoCrearPosicion genera la fila inicial en Posiciones de forma automática. Si requiere aprobación, el estado queda en Pendiente y el participante recibe una notificación informando que su solicitud está siendo revisada.

- **Flujo 2 - Registro de resultados (Árbitro):** El panel del árbitro muestra únicamente los partidos que le han sido asignados. Al seleccionar un partido, ingresa el score final de cada participante y opcionalmente el detalle por set (mapa, puntaje de cada lado, ganador del set). Al confirmar, el backend ejecuta `sp_RegistrarResultadoMatch` dentro de una transacción, que actualiza los scores y marca el partido como Finalizado. Los triggers `trg_CalcularPuntosAutomatica` y `trg_ActualizarTablaPosiciones` actualizan inmediatamente la tabla de posiciones. El trigger `trg_ControlCambiosScore` bloquea cualquier intento posterior de modificar el resultado.
- **Flujo 3 - Aprobación de inscripciones (Organizador):** El organizador accede a la vista de inscripciones de su torneo, que carga los participantes en estado Pendiente desde la vista `vw_InscripcionesPendientes`. Para cada inscripción puede hacer clic en "Aprobar" o "Rechazar". Al aprobar, `sp_ActualizarInscripcion` cambia el estado a Aceptado, lo que dispara `trg_AutoCrearPosicion` para crear la fila en Posiciones y `sp_InsertarNotificacion` para enviar una notificación automática al participante. Al rechazar, el estado cambia a Rechazado y se notifica igualmente.
- **Flujo 4 - Consulta de tabla de posiciones (Participante / Público):** La tabla de posiciones muestra en tiempo real el ranking de participantes de un torneo, con columnas de Posición, Nombre, PJ, PG, PE, PP, Score Favor, Score Contra, Diferencia y Puntos. Los datos se obtienen de `sp_ConsultarPosiciones`, ordenados por Puntos DESC y diferencia de score como criterio de desempate. La vista se actualiza automáticamente cada 30 segundos mediante polling. La implementación de WebSockets está identificada como mejora futura de alta prioridad para reducir el overhead de red y ofrecer actualizaciones verdaderamente en tiempo real durante partidas en curso.

Conclusiones

MatchControl representa una solución full stack robusta, académicamente fundamentada y técnicamente completa para la gestión de torneos multicategoría. El proyecto integró con éxito las

tres capas del desarrollo de software moderno —base de datos, backend y frontend— sobre una problemática real que afecta a organizadores de torneos en contextos estudiantiles, deportivos y gamer.

Las decisiones técnicas más relevantes del proyecto, y las lecciones aprendidas de cada una, son las siguientes:

- La separación entre participante individual y de equipo mediante el constraint `CHK_Unico_Tipo` garantiza integridad semántica a nivel del motor de base de datos, sin depender de validaciones en capas superiores que podrían ser eludidas. Esta decisión demuestra que las reglas de negocio críticas deben residir en la capa de datos.
- El acceso exclusivo a datos mediante stored procedures parametrizados elimina por completo el vector de inyección SQL y centraliza la lógica de acceso a datos en un único lugar, facilitando el mantenimiento y la auditoría de seguridad del sistema.
- La automatización por triggers garantiza que la tabla de posiciones, los registros de auditoría y las validaciones de integridad operen correctamente independientemente de si los cambios provienen del API, de herramientas administrativas o de scripts de mantenimiento, convirtiendo al motor de base de datos en el guardián definitivo de la consistencia.
- La implementación de NestJS con TypeScript en el backend proporcionó un código altamente organizado, con tipado estático que redujo significativamente los errores en tiempo de ejecución y facilitó la comprensión de la arquitectura modular del sistema.
- La estrategia de más de 30 índices NONCLUSTERED diseñados en función de patrones de acceso reales resulta en reducciones estimadas de hasta el 95% en lecturas lógicas para consultas críticas como el login por email y la carga de la tabla de posiciones, demostrando el impacto medible de la optimización de índices en sistemas reales.
- El análisis de concurrencia identificó escenarios reales de race conditions y deadlocks potenciales, y la implementación de niveles de aislamiento apropiados para cada módulo (SERIALIZABLE para inscripciones, SNAPSHOT para resultados en tiempo real) resuelve estos problemas de forma correcta sin sacrificar innecesariamente el rendimiento.

Como trabajo futuro, el sistema puede evolucionar incorporando notificaciones en tiempo real mediante WebSockets para reemplazar el polling actual, procesamiento de pagos de

inscripciones, un módulo de análisis predictivo de resultados basado en histórico, y una aplicación móvil nativa que consuma la misma API RESTful. Desde la perspectiva del modelo de negocio, el sistema tiene potencial para evolucionar hacia un modelo freemium con funcionalidades básicas gratuitas y capacidades avanzadas de pago, o hacia un modelo institucional orientado a universidades y ligas deportivas que requieran una solución integral de gestión de competencias.

Anexos

Diagrama Entidad Relación

[Visualizar Diagrama Entidad-Relación del Sistema](#)

Diagrama Relacional

[Visualizar Diagrama Relacional del Sistema](#)